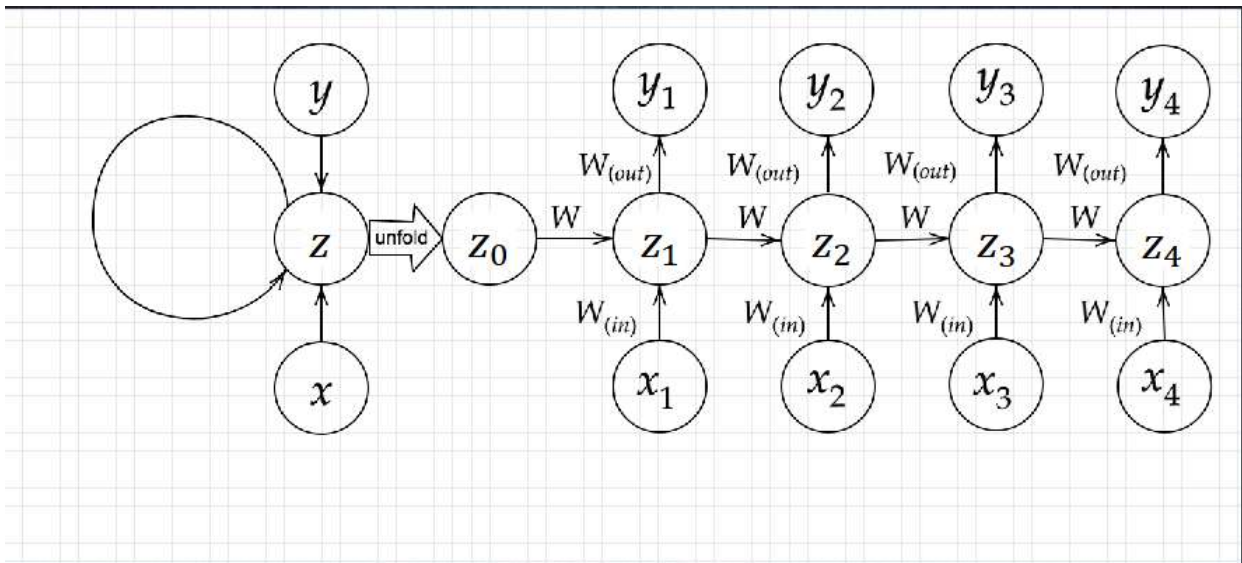


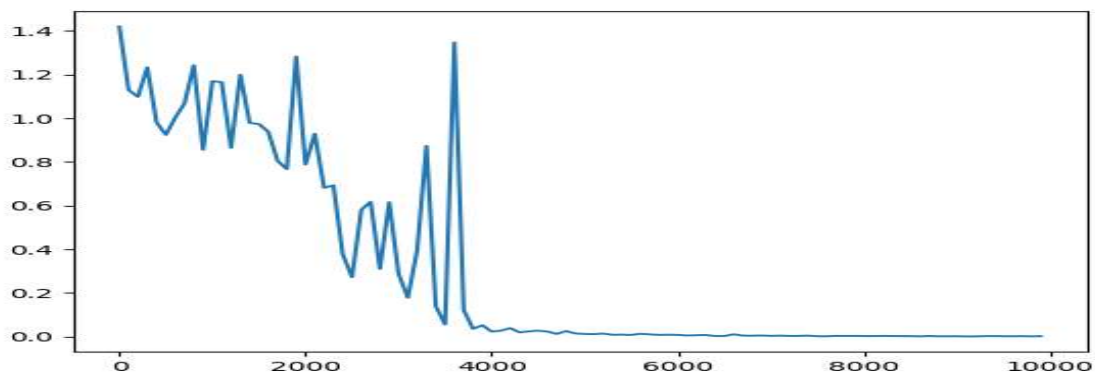
第一章 再帰型ニューラルネットワークの概念

- ・ RNN・・・時系列データに対応可能な、ニューラルネットワークである。
- ・ 時系列データ・・・時間的順序を追って一定間隔ごとに観察され、しかも時間的順序を追って一定間隔ごとに観察され、しかも依存関係が認められるようなデータの系列
- ・ 具体的な時系列データ・・・音声データ・テキストデータ…etc
- ・ RNNの全体像
出力層の出力を中間層に戻して次の入力に使う。



RNNの特徴・・・時系列モデルを扱うには、初期の状態と過去の時間t-1の状態を保持し、そこから次の時間でのtを再帰的に求める再帰構造が必要

<実践演習：3_1_simple_RNN.ipynb>



BPTT (応用学習)

BPTT・・・RNNにおいてのパラメータ調整方法の一種誤差逆伝播の一種

→誤差逆伝播の一種

誤差逆伝播法・・・計算結果 (=誤差) から微分を逆算することで、不要な再帰的計算を避けて微分を算出

$$E(\mathbf{y}) = \frac{1}{2} \sum_{j=1}^J (y_j - d_j)^2 = \frac{1}{2} \|\mathbf{y} - \mathbf{d}\|^2 \quad : \text{誤差関数} = \text{二乗誤差関数}$$
$$\mathbf{y} = \mathbf{u}^{(L)} \quad : \text{出力層の活性化関数} = \text{恒等写像}$$
$$\mathbf{u}^{(l)} = \mathbf{w}^{(l)} \mathbf{z}^{(l-1)} + \mathbf{b}^{(l)} \quad : \text{総入力 of 計算}$$

微分の連鎖率の式:

$$\frac{\partial E}{\partial w_{ji}^{(2)}} = \frac{\partial E}{\partial \mathbf{y}} \frac{\partial \mathbf{y}}{\partial \mathbf{u}} \frac{\partial \mathbf{u}}{\partial w_{ji}^{(2)}}$$

ここでは出力層の
wについて微分したい

各関数を微分

$$\frac{\partial \mathbf{y}(\mathbf{u})}{\partial \mathbf{u}} = \frac{\partial \mathbf{u}}{\partial \mathbf{u}} = 1$$

$$\frac{\partial E(\mathbf{y})}{\partial \mathbf{y}} = \frac{\partial}{\partial \mathbf{y}} \frac{1}{2} \|\mathbf{y} - \mathbf{d}\|^2 = \mathbf{y} - \mathbf{d}$$

$$\frac{\partial \mathbf{u}(\mathbf{w})}{\partial w_{ji}^{(2)}} = \frac{\partial}{\partial w_{ji}^{(2)}} (\mathbf{w}^{(l)} \mathbf{z}^{(l-1)} + \mathbf{b}^{(l)}) = \frac{\partial}{\partial w_{ji}^{(2)}} \left(\begin{bmatrix} w_{11}z_1 + \dots + w_{1i}z_i + \dots + w_{1l}z_l \\ \vdots \\ w_{j1}z_1 + \dots + w_{ji}z_i + \dots + w_{jl}z_l \\ \vdots \\ w_{l1}z_1 + \dots + w_{li}z_i + \dots + w_{ll}z_l \end{bmatrix} + \begin{bmatrix} b_1 \\ \vdots \\ b_j \\ \vdots \\ b_l \end{bmatrix} \right) = \begin{bmatrix} 0 \\ \vdots \\ z_i \\ \vdots \\ 0 \end{bmatrix}$$

BPTTの数学的記述1

$$\frac{\partial E}{\partial \mathbf{W}_{(in)}} = \frac{\partial E}{\partial \mathbf{u}^t} \left[\frac{\partial \mathbf{u}^t}{\partial \mathbf{W}_{(in)}} \right]^T = \delta^t [\mathbf{x}^t]^T$$
$$\frac{\partial E}{\partial \mathbf{W}_{(out)}} = \frac{\partial E}{\partial \mathbf{v}^t} \left[\frac{\partial \mathbf{v}^t}{\partial \mathbf{W}_{(out)}} \right]^T = \delta^{out,t} [\mathbf{z}^t]^T$$
$$\frac{\partial E}{\partial \mathbf{W}} = \frac{\partial E}{\partial \mathbf{u}^t} \left[\frac{\partial \mathbf{u}^t}{\partial \mathbf{W}} \right]^T = \delta^t [\mathbf{z}^{t-1}]^T$$
$$\frac{\partial E}{\partial \mathbf{b}} = \frac{\partial E}{\partial \mathbf{u}^t} \frac{\partial \mathbf{u}^t}{\partial \mathbf{b}} = \delta^t$$
$$\frac{\partial E}{\partial \mathbf{c}} = \frac{\partial E}{\partial \mathbf{v}^t} \frac{\partial \mathbf{v}^t}{\partial \mathbf{c}} = \delta^{out,t}$$

BPTTの数学的記述2

$$\begin{aligned} u^t &= W_{(in)}x^t + W z^{t-1} + b \\ z^t &= f(W_{(in)}x^t + W z^{t-1} + b) \\ v^t &= W_{(out)}z^t + c \\ y^t &= g(W_{(out)}z^t + c) \end{aligned}$$

BPTTの数学的記述3

$$\frac{\partial E}{\partial u^t} = \frac{\partial E}{\partial v^t} \frac{\partial v^t}{\partial u^t} = \frac{\partial E}{\partial v^t} \frac{\partial \{W_{(out)}f(u^t) + c\}}{\partial u^t} = f'(u^t) W_{(out)}^T \delta^{out,t} = \delta^t$$

$$\begin{aligned} \delta^{t-1} &= \frac{\partial E}{\partial u^{t-1}} = \frac{\partial E}{\partial u^t} \frac{\partial u^t}{\partial u^{t-1}} = \delta^t \left\{ \frac{\partial u^t}{\partial z^{t-1}} \frac{\partial z^{t-1}}{\partial u^{t-1}} \right\} = \delta^t \{ W f'(u^{t-1}) \} \\ \delta^{t-z-1} &= \delta^{t-z} \{ W f'(u^{t-z-1}) \} \end{aligned}$$

BPTTの数学的記述4

$$\begin{aligned} W_{(in)}^{t+1} &= W_{(in)}^t - \epsilon \frac{\partial E}{\partial W_{(in)}} = W_{(in)}^t - \epsilon \sum_{z=0}^{T_t} \delta^{t-z} [x^{t-z}]^T \\ W_{(out)}^{t+1} &= W_{(out)}^t - \epsilon \frac{\partial E}{\partial W_{(out)}} = W_{(out)}^t - \epsilon \delta^{out,t} [z^t]^T \\ W^{t+1} &= W^t - \epsilon \frac{\partial E}{\partial W} = W_{(in)}^t - \epsilon \sum_{z=0}^{T_t} \delta^{t-z} [z^{t-z-1}]^T \\ b^{t+1} &= b^t - \epsilon \frac{\partial E}{\partial b} = b^t - \epsilon \sum_{z=0}^{T_t} \delta^{t-z} \\ c^{t+1} &= c^t - \epsilon \frac{\partial E}{\partial c} = c^t - \epsilon \delta^{out,t} \end{aligned}$$

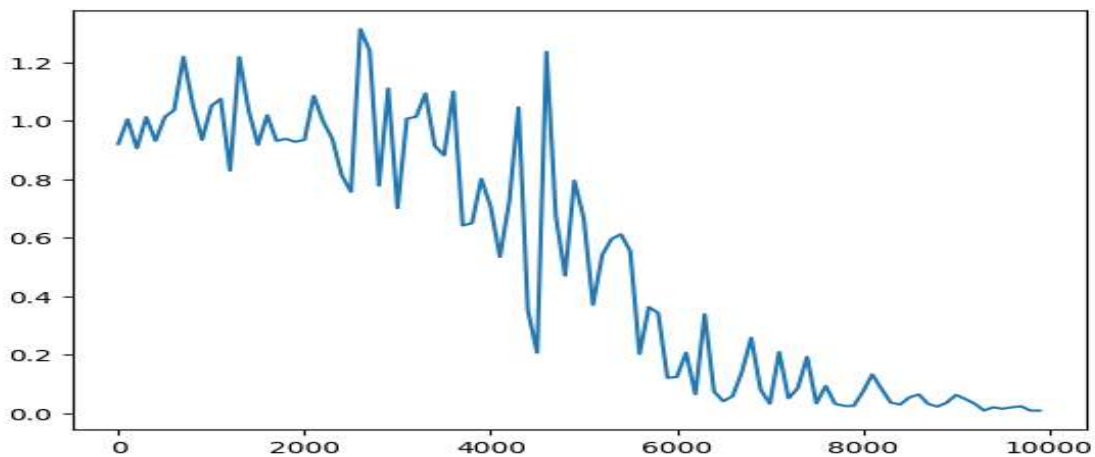
・ BPTTの全体像

$$\begin{aligned} E^t &= \text{loss}(y^t, d^t) \\ &= \text{loss}(g(W_{(out)}z^t + c), d^t) \\ &= \text{loss}(g(W_{(out)}\underline{f(W_{(in)}x^t + W z^{t-1} + b)} + c), d^t) \end{aligned}$$



$$\begin{aligned} &W_{(in)}x^t + W z^{t-1} + b \\ &W_{(in)}x^t + W f(u^{t-1}) + b \\ &W_{(in)}x^t + W f(W_{(in)}x^{t-1} + W z^{t-2} + b) + b \end{aligned}$$

< 実践演習 : 3_1_simple_RNN_after.ipynb >



第二章 LSTM

RNNの課題・・・時系列を遡れば遡るほど、勾配が消失していく。

→長い時系列の学習が困難。

・ 解決策：勾配消失の解決方法とは別で、構造自体を変えて解決したものがLSTM。

・ 勾配消失問題

誤差逆伝播法が下位層に進んでいくに連れて、勾配がどんどん緩やかになっていく。

そのため、勾配降下法による、更新では下位層のパラメータはほとんど変わらず

訓練は最適値に収束しなくなる。

・ 勾配消失問題の理由

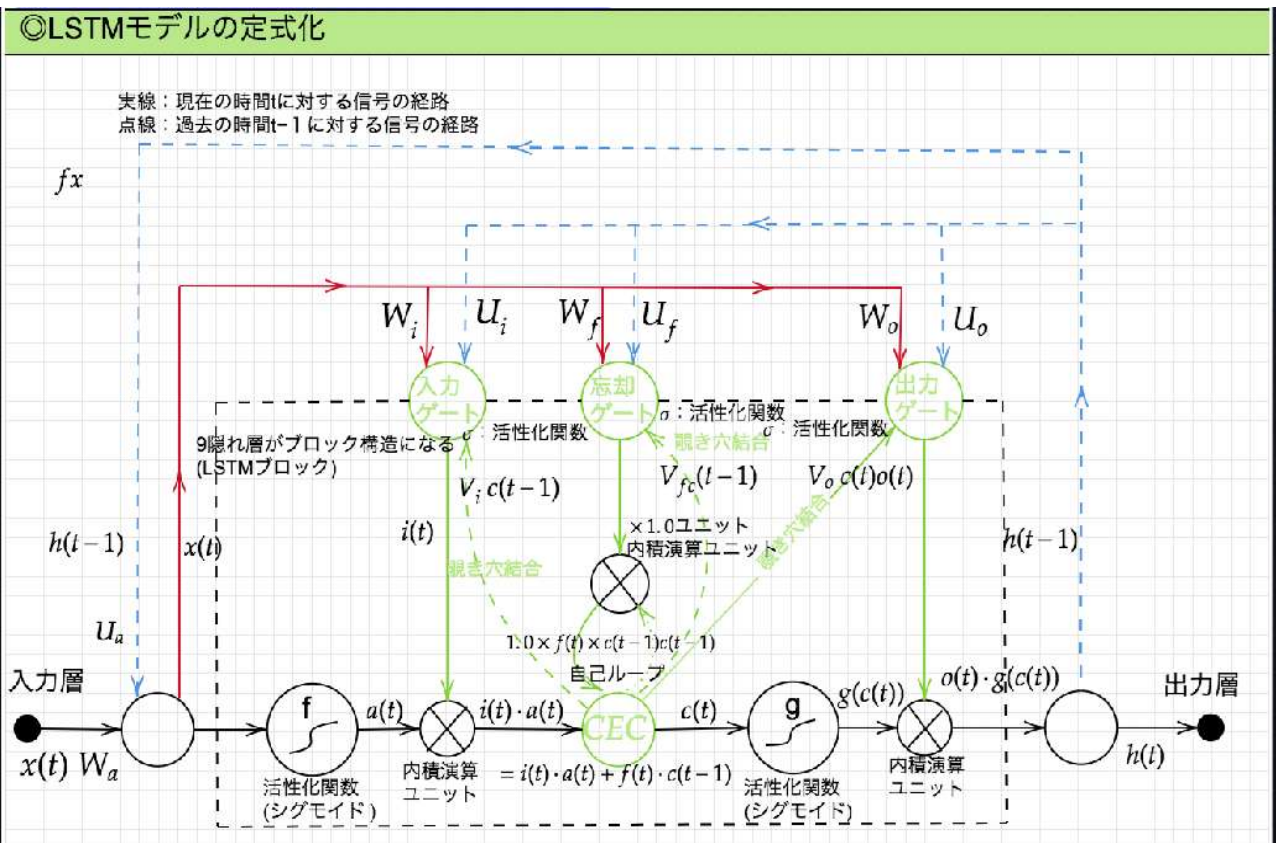
中間層が増えれば増えるほど微分の組み合わせが増え、調整量が少なくなるため

・ 活性化関数：シグモイド関数・・・勾配消失問題を引き起こす関数。

・ 勾配爆発・・・勾配が、層を逆伝播するごとに指数関数的に大きくなっていくこと

恒等関数が勾配爆発を引き起こす

・ LSTMの全体図



- ・ CEC・・・RNNでいう中間層。勾配消失および勾配爆発の解決方法として
勾配が、1であれば解決できる。そのために導入。覚える機能のみ
 - ・ 課題：入力データについて、時間依存度に関係なく重みが一律である。
- ニューラルネットワークの学習特性が無いということ

- ・ 入力ゲートと出力ゲート
 - ・ 入力・出力ゲートの役割
- 入力・出力ゲートを追加することで、それぞれのゲートへの入力値の重みを、
重み行列 W, U で可変可能とする
- CECの課題を解決。

- ・ LSTMの現状
- CECは、過去の情報が全て保管されている
- ・ 課題：過去の情報が要らなくなった場合、削除することはできず保管され続ける。
- 解決策：過去の情報が要らなくなった場合、そのタイミングで情報を忘却する機能が必要。
- 忘却ゲートの誕生

- ・ 覗き穴結合・・・CEC自身の値に、重み行列を介して伝播可能にした構造。

第三章 GRU

課題：LSTMでは、パラメータ数が多く、計算負荷が高くなる問題があった。

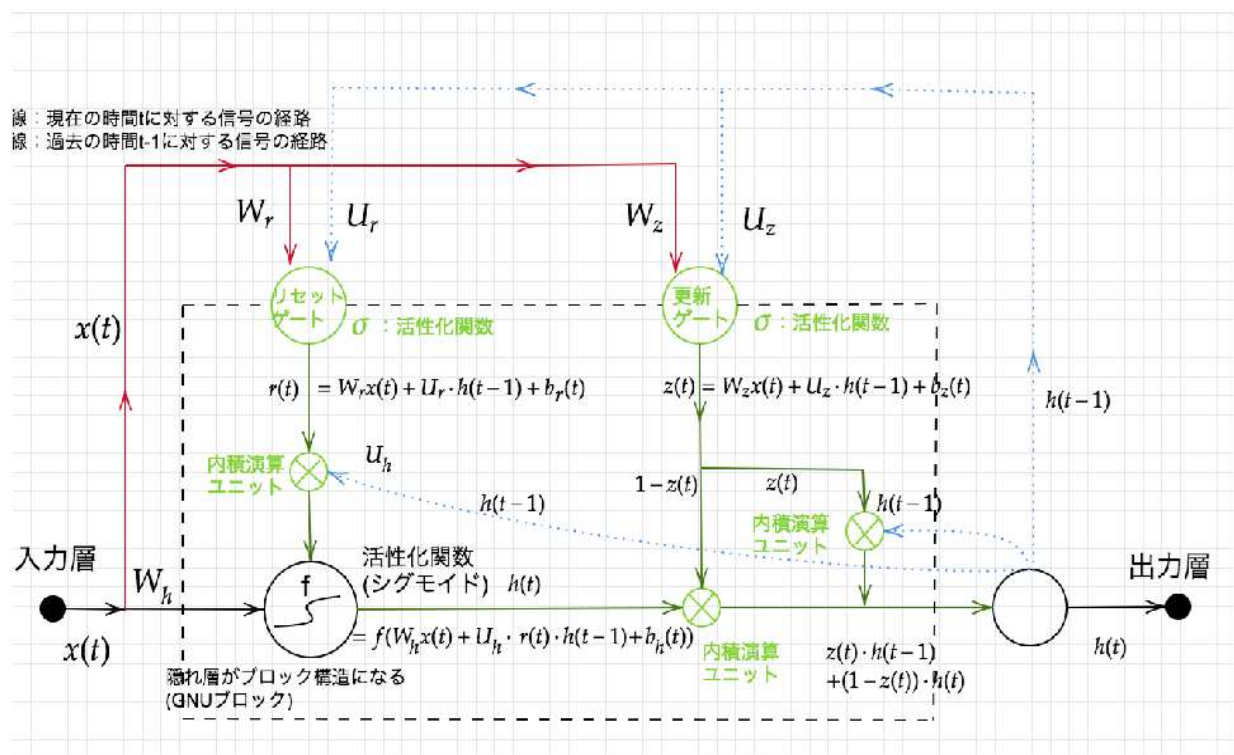
→GRUにて対応

GRU・・・パラメータを大幅に削減し、精度は同等またはそれ以上が望める

様になった構造

メリット：計算負荷が低い。

GRUの全体像



< 実践演習 : predict_word.ipynb >

```
In = Language()

# 学習済みのパラメータをロード
In.load_weights("./data_for_predict/predict_model")

# 保存したモデルを使って単語の予測をする
In.predict("some of them looks like")

award : 1.30829744e-14
head : 1.3081603e-14
agency : 1.3079782e-14
sailors : 1.3079307e-14
dogs : 1.3078559e-14
skirt : 1.30777355e-14
tapping : 1.3076863e-14
```

< 実践演習 : 3_4_spoken_digit.ipynb >

[16]
✓ 11 秒

```
model_3.compile(
    loss=tf.keras.losses.SparseCategoricalCrossentropy(),
    optimizer=tf.keras.optimizers.Adam(),
    metrics=['accuracy']
)

model_3.fit(
    dataset_prep_train,
    validation_data=dataset_prep_valid,
)
```

✓ ... 219/219 ————— 11s 37ms/step - accuracy: 0.0910 - loss: 2.3913 - val_accuracy: 0.1253 - val_loss: 2.2910
<keras.src.callbacks.history.History at 0x7bc3747ccdd0>

第四章 双方向RNN

双方向RNN・・・過去の情報だけでなく、未来の情報を加味することで、精度を向上させるためのモデル

実用例：文章の推敲や、機械翻訳等

< 実践演習：3_4_spoken_digit.ipynb >

```
[20] 12 秒
✓ model_5.compile(
    loss=tf.keras.losses.SparseCategoricalCrossentropy(),
    optimizer=tf.keras.optimizers.Adam(clipvalue=0.5),
    metrics=['accuracy']
)

model_5.fit(
    dataset_prep_train,
    validation_data=dataset_prep_valid,
)

✓ ... 219/219 ————— 12s 32ms/step - accuracy: 0.1011 - loss: 2.3536 - val_accuracy: 0.0800 - val_loss: 2.2915
<keras.src.callbacks.history.History at 0x7bc360310ad0>
```

第五章 Seq2seq

- Seq2seq・・・Encoder-Decoderモデルの一種

Seq2seqの具体的な用途：機械対話や、機械翻訳などに使用

- Encoder RNN・・・ユーザーがインプットしたテキストデータを、単語等のトークンに区切って渡す構造。
- Taking：文章を単語等のトークン毎に分割し、トークンごとのIDに分割する。
- Embedding：IDから、そのトークンを表す分散表現ベクトルに変換。
- Encoder RNN:ベクトルを順番にRNNに入力していく。

- Encoder RNN処理手順

- vec1をRNNに入力し、hidden stateを出力。

このhidden stateと次の入力vec2をまたRNNに入力してきたhidden stateを出力という流れを繰り返す。

- 最後のvecを入れたときのhiddenstateをfinal stateとしてとっておく。

このfinalstateがthought vectorと呼ばれ、入力した文の意味を表すベクトルとなる。

- Decoder RNN・・・システムがアウトプットデータを、単語等のトークンごとに生成する構造

- Decoder RNNの処理

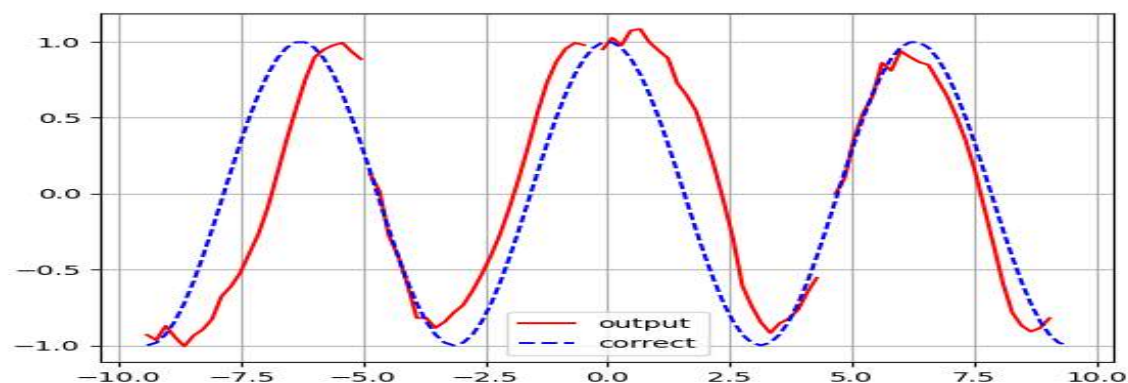
1.Decoder RNN: Encoder RNN の final state (thought vector) から、各token の生成確率を出力するfinal state をDecoder RNN のinitial state として設定し、Embedding を入力。

2.Sampling: 生成確率にもとづいてtoken をランダムに選ぶ

3.Embedding: 2で選ばれた token を Embedding して Decoder RNN への次の入力とする。

4.Detokenize: 1 -3 を繰り返し、 2で得られた token を文字列に直す。

< 実践演習：3_5_Seq2Seq(Encoder-Decoder)_sin-cos.ipynb >



- ・ Seq2seqの課題

課題：一問一答しかできない

→問に対して文脈も何もなく、ただ応答が行われ続ける。

→HREDの誕生

- ・ HRED・・・過去n-1 個の発話から次の発話を生成する。

→Seq2seqでは、会話の文脈無視で、応答がなされたが、HREDでは、前の単語の流れに即して応答されるため、より人間らしい文章が生成

→Seq2Seq+ Context RNN

- ・ Context RNN: Encoder のまとめた各文章の系列をまとめて、これまでの会話コンテキスト全体を表すベクトルに変換する構造。

→過去の発話の履歴を加味した返答をできる。

- ・ HREDの課題：HRED は確率的な多様性が字面にしかなく、会話の「流れ」のような多様性が無い。

→同じコンテキスト（発話リスト）を与えられても、答えの内容が毎回会話話の流れとしては同じものしか出せない。

HRED は短く情報量に乏しい答えをしがちである。

→短いよくある答えを学ぶ傾向がある。

- ・ VHRED・・・HREDに、VAEの潜在変数の概念を追加したもの。

→HREDの課題を、VAEの潜在変数の概念を追加することで解決した構造。

- ・ VAE・・・入力データの背後にある潜在的な確率分布を学習することで、新たなデータ生成や特徴表現の獲得が可能

- ・ オートエンコーダ・・・教師なし学習の一つ。そのため学習時の入力データは訓練データのみで教師データは利用しない。

- ・ オートエンコーダ具体例：

MNISTの場合、28x28の数字の画像を入れて、同じ画像を出力するニューラルネットワーク

- ・ オートエンコーダ構造説明

入力データから潜在変数 z に変換するニューラルネットワークをEncoder

逆に潜在変数 z をインプットとして元画像を復元するニューラルネットワークをDecoder。

メリット：次元削減が行えること。

・オートエンコーダの構造図



オートエンコーダとVAEの違い

	オートエンコーダ	VAE
潜在変数	・連続値で、ユークリッド空間上に分布しない。	・正規分布に従う確率分布としてモデル化される。
潜在変数の意味	・直線的に補間できるが、その意味や連続性は保証されない。	・統計的に意味があり、連続的であるため潜在空間内でのサンプリングや補間が意味を持つ。

・VAEのメリット

メリット	概要
数学的に扱いやすい	平均と分散により、簡単に統計的性質を調整できる。
連続性を持つ	潜在空間内の異なる点を、滑らかに連続的に補間できる。

- ・ VAEの学習・・・VAEの学習は、入力データに対する条件付き確率分布を調整し尤度を最大にすることを目標として行われる。
- ・ ELBO（変分下限）・・・尤度関数を最大化する手法
- ・ Reparametrization trick・・・確率分布からのサンプリングを微分可能な操作に変換する手法
- ・ Reparametrization trickの役割
- ・ Reparametrization trickの役割 VAEでは、エンコーダが潜在変数の確率分布のパラメータである「平均」と「分散」を出力し、デコーダがその分布からサンプリングした潜在変数を用いてデータを生成
- ・ 正規分布からのサンプリングには乱数が介入するため、そのままでは微分不可能
→Reparametrization trickを使用することで、サンプリングの操作を微分可能

- ・ Denoising Autoencoder (DAE) . . . オートエンコーダの一種で、主にノイズの除去に焦点を当てたモデル。

- ・ VAEでは、潜在空間内に各ラベルのデータが正規分布に従って分布する

- ・ 調整した潜在変数から画像の生成を行うことで、特定の部分が強調されたデータを得ることができる。

→調整するステップ

1. 潜在変数の取得：生成したい画像を用いてVAEを学習し、潜在変数のマッピングを取得する。
2. 潜在変数の特定の成分の変更：潜在変数の特徴を変更し、特定の部分の強調を行う。
3. 潜在変数から画像の作成：調整した潜在変数から画像の生成を行う。

第六章 word2vec

- ・ 課題:RNNでは、単語のような可変長の文字列をNNに与えることはできない。
→固定長形式で単語を表す必要がある。
- ・ word2vec・・・学習データからボキャブラリを作成
- ・ one-hot ベクトル・・・入力層には辞書の単語数だけベクトル（0と1）が入力
- ・ メリット：大規模データの分散表現の学習が、現実的な計算速度とメモリ量で実現可能
→ボキャブラリ数×単語ベクトルの次元数の重み行列が作成
- ・ 自然言語処理
- ・ 自然言語処理Word Embedding（単語の埋め込み）
→文章中の単語の意味や文法を反映するように単語をベクトルに変換する手法
→密な固定長のベクトル
→分布仮説：文章中の単語の意味は周囲の単語によって形成
→特徴①類似性：同じような意味の単語は同じようなベクトルになる性質
→加法構成性：単語間のパターンを反映
- ・ Word Embedding（単語の埋め込み）には、推論ベースのニューラルネットワークを用いる手法（Word2Vec）とカウントベースの手法がある。
- ・ 推論ベースの手法（Word2Vec）
→n-gramによるトークン化
→ニューラルネットワークによる推論
→CBOWとSkip Gram：Word2Vecのモデル
→ネガティブサンプリング：Word2Vecの効率化手法
- ・ 特異値分解を用いたカウントベースの手法
→共起数（※）をカウント
→特異値分解による次元削減
※テキストデータ（文書や発話など）において、特定の2つ以上の単語やフレーズが一緒に現れる頻度のこと
- ・ CBOW・・・周囲の複数の単語（コンテキスト）から対処いうの単語（ターゲット）を予測するモデル
- ・ Skip gram・・・対象の単語（ターゲット）から周囲のh区数の単語（コンテキスト）を予測するモデル。

- ・ネガティブサンプリング・・・Word 2 Vecの学習を効率化する技術
- ・Word2Vecの問題点：辞書に含まれる単語の数 n に応じて計算（行列計算やソフトマックス関数）が多くなる
→正解ラベルの単語（ポジティブ）と少数のネガティブな単語だけサンプリングしそれぞれ2値分類タスクをお行う。
- ・One-hotベクトルとWord Embeddingの違い
 - ・One-hot・・・疎なベクトル
 - ・Word Embedding・・・密なベクトル、類似性、加法構成性

第七章 AttentionMechanism

課題:seq2seq の問題は長い文章への対応が難しい。

seq2seq では、2単語でも、100単語でも、固定次元ベクトルの中に入力しなければならない。

解決策：文章が長くなるほどそのシーケンスの内部表現の次元も大きくなっていく

仕組みが必要

→AttentionMechanism

「入力と出力のどの単語が関連しているのか」の関連度を学習する仕組み

第八章 VQ-MAE

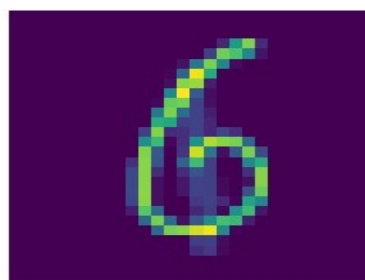
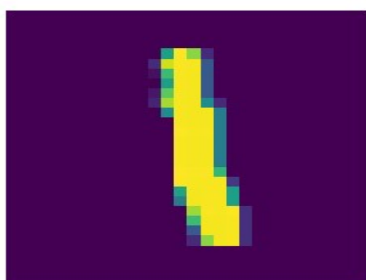
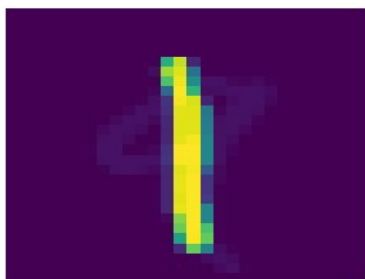
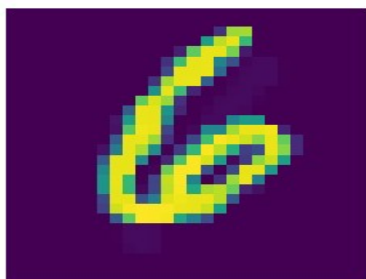
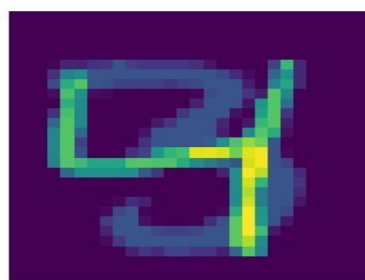
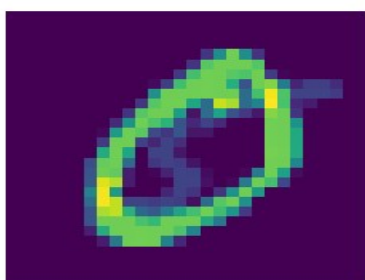
VQ-MAEは「ベクトル量子化（VQ）＋マスク付きオートエンコーダ（MAE）」を組み合わせた自己教師あり学習モデル。

- ・ 1. VQ（Vector Quantization：ベクトル量子化）
音声をそのまま扱うのではなく、- 「コードブック」と呼ばれる辞書の中の“番号”に変換する仕組み
- ・ 2. MAE（Masked AutoEncoder：マスク付きオートエンコーダ）
- 入力の一部を隠して（マスクして）- 隠された部分を復元するように学習するモデル。

第九章 フレームワーク演習

- ・ data-augmentation
- ・ データの水増し・・・実際には無限に多くの画像を収集することは難しい場合があり
新しいデータを生成する方法が必要。
→「データの水増し」または「データ拡張」
→画像を反転処理・回転処理する。

<実践演習：3_6_data_augmentation_with_tf_add2.ipynb>



['類似', 'する', 'データ', 'を', '生成', 'する', '記事', 'を', '書い', 'て', 'ます', '。']

['類似', 'する', 'データ', ' ', '生成', 'する', '記事', ' ', '書く', ' ', ' ', ' ']

類似 → 似る

する → 行う

原文：類似するデータを生成する記事を書いています。

変換後：似る行うデータを生成する記事を書いています。

['似る', '行う', 'データ', 'を', '生成', 'する', '記事', 'を', '書い', 'て', 'ます', '。']

random_insertion(raw_words, original_words, 2)

... 挿入する単語：酷似

挿入する単語：とり熟す

原文：類似するデータを生成する記事を書いています。

変換後：酷似類似するデータを生成する記事を書いていますとり熟す。

['酷似',
'類似',
'する',
'データ',
'を',
'生成',
'する',
'記事',
'を',
'書い',
'て',
'ます',
'とり熟す',
'。']

random_deletion(raw_words, 0.05)

... 削除：を

原文：類似するデータを生成する記事を書いています。

の出力アクション 類似するデータを生成する記事書いてます。

['類似', 'する', 'データ', 'を', '生成', 'する', '記事', '書い', 'て', 'ます', '。']

random_swap(raw_words, 2)

... 記事 ⇄ ます

類似 ⇄ て

原文：類似するデータを生成する記事を書いています。

変換後：てするデータを生成するますを書い類似記事。

['て', 'する', 'データ', 'を', '生成', 'する', 'ます', 'を', '書い', '類似', '記事', '。']

```
# 原文から5つの文章を生成
eda(text, alpha_sr=0.05, alpha_ri=0.05, alpha_rs=0.05, p_rd=0.05, num_aug=5)
```

```
... 類似 → 類縁
原文：類似するデータを生成する記事を書いています。
変換後：類縁するデータを生成する記事を書いています。

挿入する単語：生産
原文：類似するデータを生成する記事を書いています。
変換後：類似するデータを生成する記事を生産書いてます。

書き ⇄ て
原文：類似するデータを生成する記事を書いています。
変換後：類似するデータを生成する記事を書いて書きます。

削除：を
原文：類似するデータを生成する記事を書いています。
変換後：類似するデータ生成する記事を書いています。

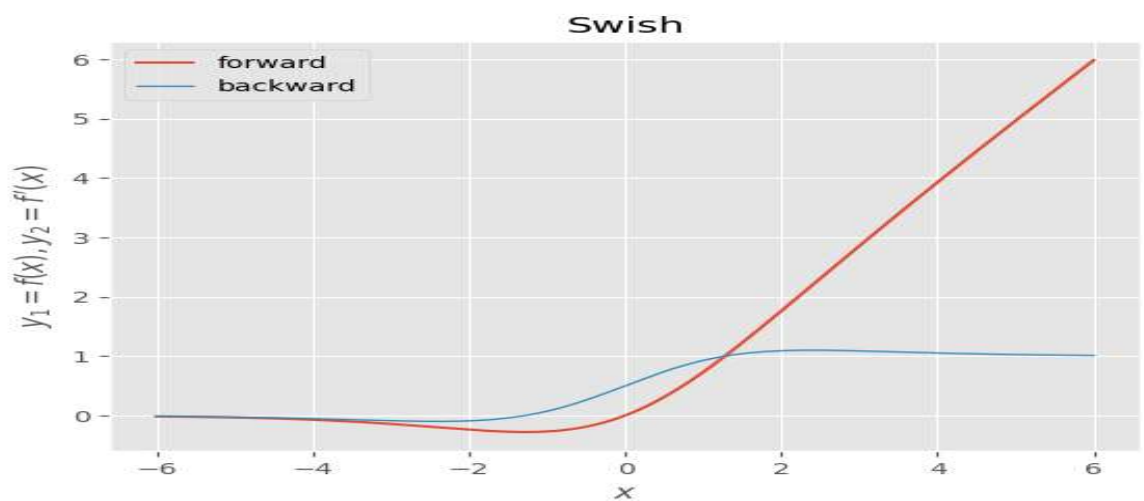
['類似するデータを生成する記事を生産書いてます。',
'類縁するデータを生成する記事を書いています。',
'類似するデータ生成する記事を書いています。',
'類似するデータを生成する記事を書いて書きます。',
'類似するデータを生成する記事を書いています。']
```

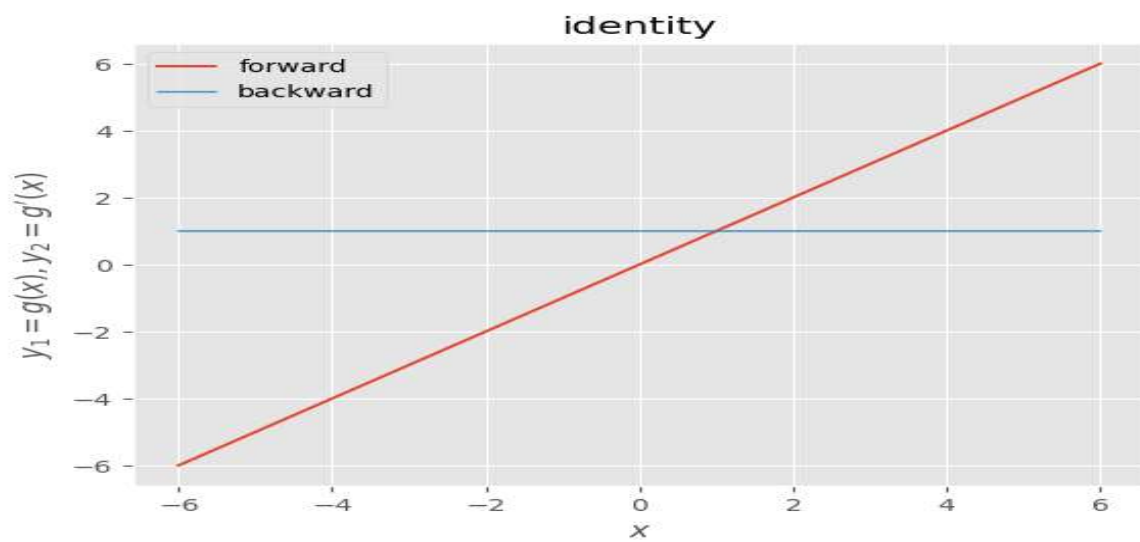
・ activate_functions (活性化関数)

→関数とは、渡された入力値 (引数) を元に、何らかの処理を施し、新たな出力値を生成・返却するもの

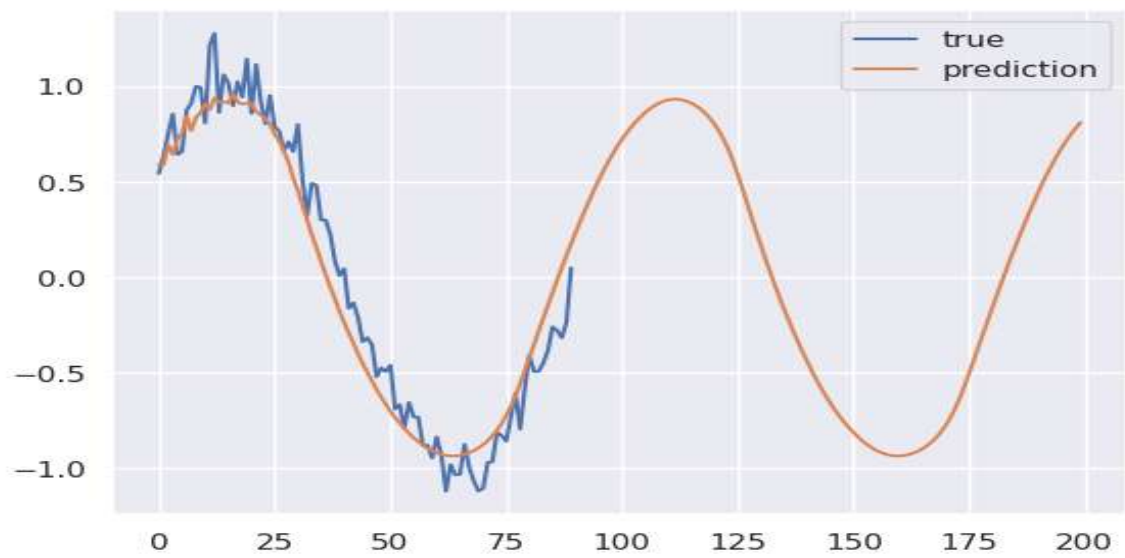
・ 活性化関数について・・・非線形な変換を行う際に用いられる関数

< 実践演習：3_7_activation_functions.ipynb >

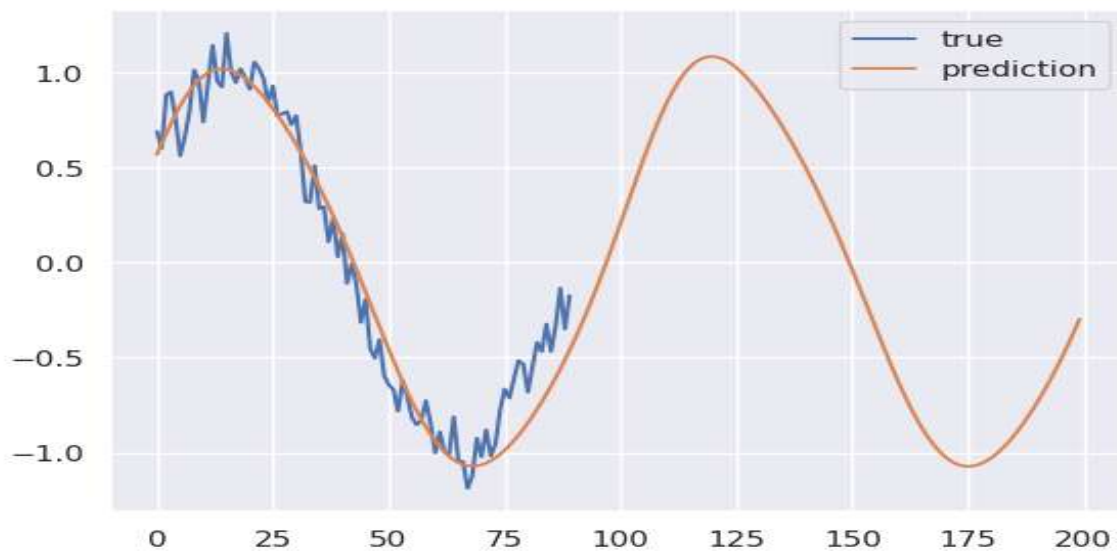




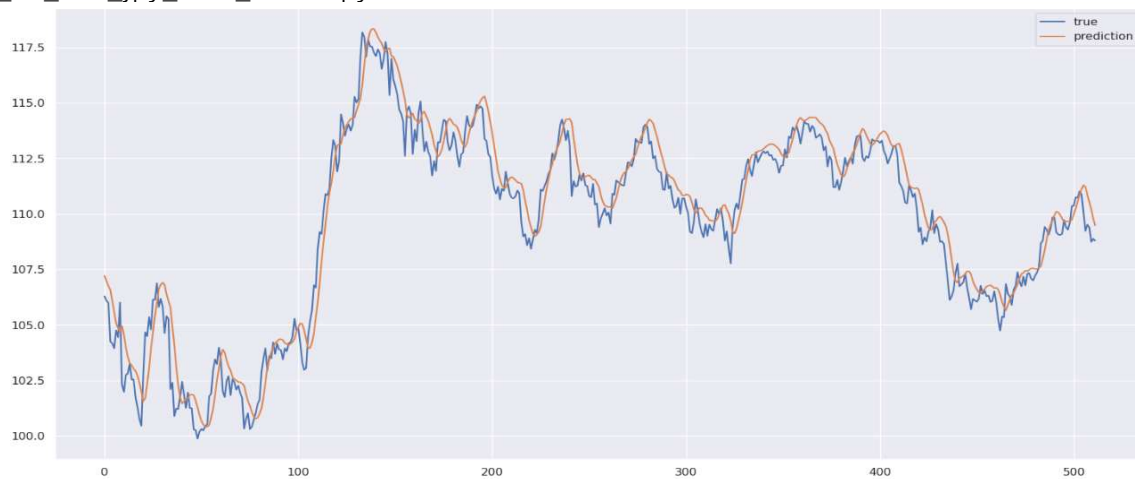
< 実践演習 : 3_8_rnn_keras.ipynb >



<実践演習：3_9_rnn_torch.ipynb>



<3_10_usd_jpy_lstm_keras.ipynb>



<3_11_nlp-basic.ipynb>

```
... 1/1 _____ 0s 198ms/step
1/1 _____ 0s 31ms/step
1/1 _____ 0s 33ms/step
1/1 _____ 0s 33ms/step
1/1 _____ 0s 33ms/step
1/1 _____ 0s 32ms/step
1/1 _____ 0s 35ms/step
1/1 _____ 0s 31ms/step
1/1 _____ 0s 31ms/step
1/1 _____ 0s 33ms/step
1/1 _____ 0s 35ms/step
1/1 _____ 0s 31ms/step
1/1 _____ 0s 33ms/step
1/1 _____ 0s 31ms/step
1/1 _____ 0s 33ms/step
1/1 _____ 0s 37ms/step
1/1 _____ 0s 32ms/step
1/1 _____ 0s 30ms/step
1/1 _____ 0s 31ms/step
1/1 _____ 0s 32ms/step
1/1 _____ 0s 30ms/step
この季節には京都に行って|いる|人|も|いる|と|思う|の|でしょう|じゃ|、|「|実際|に|とっ|て|いる|と|は|限ら|ない
```

< 実践演習 : 3_12_rnn_np.ipynb >

